

Requirements analysis

Step 1: Requirement Analysis

Project Overview

Course: B.Tech. CSE (Semester 5)

Subject: System Design (TCS-504)

Project: Movie Ticket Booking System

1. Problem Statement

Build a **movie ticket booking system** for a single cinema (like PVR or INOX) as a menu-driven C++ console program.

A customer should be able to:

- See which movies are playing.
- Pick a show.
- See which seats are free.
- Book seats and make a payment.
- Get a ticket and cancel a booking.

2. Scope - The 8 Core Features

Feature ID	Feature Name	Description
F1	List Movies	List all movies currently playing with language and duration.
F2	List Shows	For a chosen movie, list its shows (screen and start time).
F3	Display Seat Layout	For a chosen show, display the seat layout showing available and booked seats
F4	Book Seats	Book one or more seats for a show (reject an already-booked seat).
F5	Tiered Pricing	Calculate price based on seat type (Silver: Rs.150, Gold: Rs.250).

Feature ID	Feature Name	Description
F6	Payment Processing	Pay by UPI, Card, or Cash. A failed payment must not confirm the booking.
F7	Print Ticket	Print a ticket showing booking ID, movie, screen, time, seats, and amount.
F8	Cancel Booking	Cancel a booking so the seats become available again.

3. Functional Requirements (FR)

What are Functional Requirements?

These are the exact steps, inputs, and outputs required for each feature to work correctly in the system.

FR1: List All Movies Currently Playing

- **Description:** The system displays a list of all movies currently scheduled at the cinema.
- **Input:** User selects menu option 1 (View Movies).
- **Processing:** Get and format all movie details (movie ID, title, language, duration).
- **Expected Output:**

```
[1] 3 Idiots      | Language: Hindi   | Duration: 170 min
[2] Interstellar  | Language: English | Duration: 169 min
```

Edge Cases

- If no movies are playing, show: "No movies currently playing."

FR2: List Shows for a Chosen Movie

- **Description:** For a chosen movie, the system lists all scheduled shows (screen and start time).
- **Input:** Movie choice number (e.g., 1).
- **Processing:**
 1. Check if the chosen number is a valid movie.
 2. Get and display all shows for that movie.
- **Expected Output:**

```
--- Shows for '3 Idiots' ---  
[1] Screen-1 | 06:00 PM  
[2] Screen-2 | 09:00 PM
```

Edge Cases

- If the user enters a wrong number, show "Invalid selection" and ask again without crashing.

FR3: Display Seat Layout with Availability

- **Description:** For a chosen show, the system shows the seating arrangement categorized by seat type, clearly showing which seats are available and which are booked.
- **Input:** Show choice number (e.g., 1).
- **Processing:**
 1. Get all seats and their current status for the selected show.
 2. Group seats by type (Silver, Gold, Platinum).
 3. Show available seats as [] and booked seats as [X] .
- **Expected Output:**

```
SCREEN-1 | 06:00 PM | 3 Idiots  
SILVER   (Rs. 150) : A1[ ]  A2[X]  A3[ ]  A4[ ]  
GOLD     (Rs. 250) : B1[ ]  B2[ ]  B3[X]  B4[ ]  
PLATINUM (Rs. 400) : C1[ ]  C2[ ]  C3[ ]  C4[ ]  
  
Legend: [ ] = AVAILABLE, [X] = BOOKED
```

Edge Cases

- Any seat booked in an earlier transaction must show as [X] .

FR4: Book One or More Seats (Conflict Checking)

- **Description:** The system allows booking one or multiple seats for a show, rejecting any already-booked or fake seats.
- **Input:** Comma-separated seat names (e.g., A1, B2).
- **Processing:**
 1. Read the input into individual seats.

2. For every seat:
 - Check if the seat actually exists.
 - Check if the seat is available.
3. If ANY seat is invalid or already booked:
 - Reject the entire booking.
 - Do not change any seat status.
 - Show an error explaining which seat caused the issue.
4. If ALL seats are valid and available:
 - Move to pricing and payment.

Expected Outputs

- **On Success:** Confirmation of selected seats and move to payment.
- **On Failure:** "Error: Seat A2 is already booked. Booking rejected."

Edge Cases

- **Fake seat** (e.g., Z99): Show "Seat Z99 does not exist."
- **Duplicate input** (e.g., A1, A1): Reject duplicate selection.

FR5: Price the Booking by Seat Type

- **Description:** The system calculates the total booking price based on the fixed rates of each chosen seat's type.
- **Pricing Rates:**
 - **Silver** = Rs. 150
 - **Gold** = Rs. 250
 - **Platinum** = Rs. 400
- **Input:** List of validated seats.
- **Processing:** Add up the price for each selected seat.
- **Expected Output:**

```
A1  SILVER  Rs. 150
B2  GOLD    Rs. 250
-----
TOTAL AMOUNT Rs. 400
```

Edge Cases

- Correctly add up mixed seat types (e.g., 1 Silver + 1 Platinum = Rs. 150 + Rs. 400 = Rs. 550).
-

FR6: Pay by UPI, Card, or Cash

- **Description:** The system handles payment via UPI, Card, or Cash. Only a successful payment confirms the booking. A failed payment cancels the process and frees the seats.
- **Input:** Payment choice (1. UPI, 2. Card, 3. Cash) and simulated payment confirmation.
- **Processing:**
 1. Pass the transaction to the Payment module.
 2. Execute the payment for the total amount.
 3. If payment succeeds:
 - Change seat status to booked.
 - Generate a unique Booking ID (e.g., BK1001).
 - Save the booking record.
 4. If payment fails (or is cancelled by user):
 - Cancel the booking.
 - Release all seats back to available.

Expected Outputs

- **Success:** "[UPI] Rs. 400 paid successfully. Booking Confirmed!"
- **Failure:** "Payment failed. Booking not confirmed. Seats released."

Edge Cases

- A failed payment must keep the seat layout exactly as it was before the booking attempt.
-

FR7: Print a Formatted Ticket

- **Description:** The system formats and prints a detailed ticket receipt when a booking is confirmed.
- **Input:** Confirmed Booking record.
- **Processing:** Extract booking ID, movie title, screen name, showtime, booked seat numbers, and total paid amount.
- **Expected Output:**

```
=====
                        BOOKING TICKET
=====
Booking ID : BK1001
Movie      : 3 Idiots
Screen     : Screen-1 (06:00 PM)
Seats      : A1, B2
Amount     : Rs. 400
Status     : CONFIRMED
=====
```

Edge Cases

- Ticket printing must be done by a dedicated printing class, keeping display logic separate from core data.

FR8: Cancel a Booking and Release Seats

- **Description:** The customer can cancel an existing booking using its Booking ID, immediately making the seats available again.
- **Input:** Booking ID string (e.g., BK1001).
- **Processing:**
 1. Search for the Booking ID in the records.
 2. If found and active:
 - Mark booking status as cancelled.
 - Reset all associated seats in the show to available.
 3. If not found or already cancelled:
 - Display an error message.

Expected Outputs

- **Success:** "Booking BK1001 cancelled successfully. Seats A1, B2 are now AVAILABLE."
- **Failure:** "Error: Booking ID BK1001 not found or already cancelled."

Edge Cases

- Checking the seat layout right after a cancellation must show those seats as [] .

4. Non-Functional Requirements (NFR)

What are Non-Functional Requirements?

These are the rules that dictate *how* the system should be built to be clean, safe, and easy to update.

NFR1: Modularity and Code Structure

- The system must follow strict modularity with **one class per file**.
- Core data classes (`Movie` , `Screen` , `Seat` , `Show` , `ShowSeat` , `Booking`) must be kept separate from service classes (`PriceCalculator` , `Payment` , `TicketPrinter` , `BookingService`).

NFR2: Easy to Expand

- The payment module must use an abstract base class (`Payment`).
- Adding a new payment method (like NetBanking) should only require adding a new class file, without changing the existing `BookingService` code.
- Adding new seat types must not require changes to the layout drawing logic.

NFR3: Handling Errors Gracefully

- The application must **never crash** on wrong, out-of-range, or unexpected user inputs.
- If the user types a letter instead of a number in a menu, the system should clear the error and ask again safely.
- Seat input must be case-insensitive (e.g., `"a1"` , `"b2"` automatically reads as `"A1"` , `"B2"`).

NFR4: Transaction Safety

- Booking must be fully complete or not complete at all. You cannot have a situation where only half the requested seats get booked.
- Memory states (Available to Booked) must remain consistent at all times.

5. Summary Table

Req ID	Requirement	Success Verification Criterion
FR1	List Movies	Displays all initialized movies with language and duration.
FR2	List Shows	Displays only shows scheduled for the selected movie.

Req ID	Requirement	Success Verification Criterion
FR3	Seat Layout	Matrix shows correct [] (Available) and [X] (Booked).
FR4	Book Seats	Books available seats; immediately rejects booked seats.
FR5	Tiered Pricing	Correct calculation: Silver (Rs. 150), Gold (Rs. 250), Platinum (Rs. 400).
FR6	Payment	Payment success confirms booking; failure restores seats.
FR7	Print Ticket	Formatted receipt with ID, movie, screen, seats, total amount.
FR8	Cancel Booking	Cancellation frees seats, which show as [] on next view.
NFR1	Modularity	One class per file; clear single responsibilities.
NFR2	Extensibility	Abstract Payment interface allows adding NetBanking easily.
NFR3	Robustness	Application recovers gracefully from invalid inputs without crashing.
NFR4	Atomicity	No partial bookings or orphaned reserved seats on payment drops.

Next Step: Proceed to [NOUN_VERB_ANALYSIS](#) for the Object-Oriented Design choices.